

THE CORPORATE HEIST

Solution Documentation

Scrum-Driven Delivery for Nightingale Top-5% Hire Prediction

Team	Sprint Duration	Framework	Date
Team Spirit	2 Hours (Time-boxed)	Scrum + CRISP-DM	19 Sep 2026

1. Product Vision & Goal

Product Vision: Identify and rank the 500 applicants (top 5% of 10,000) in Nightingale's current cycle (the Vault) who will become the company's best hires, so Kestrel Talent can reach them first.

Sprint Goal: Deliver a validated, ranked shortlist of exactly 500 candidate_ids, supported by clean code and documentation that reflects the insider debrief and historical patterns.

2. Scrum Roles (Mapped)

Role	Mapping in this Project
Product Owner	Nightingale hiring managers (via insider voice note) + competition brief. Defined the "Definition of Done": ranked 500, no fakes, no duplicates.
Scrum Master	Process facilitator – enforced time-box, prioritised backlog items (cleaning → modelling → rules → ranking), removed blockers.
Development Team	Data scientists / engineers who owned parsing, feature engineering, model training, rule engine and packaging.

3. Product Backlog (Prioritised)

Ordered by business value and risk (MoSCoW + effort).

#	User Story / Backlog Item	Priority	Status
1	As a talent scout I need clean, standardised candidate features so that the model is not misled by dirty data.	Must	Done
2	As a hiring committee I need fabricated and duplicate profiles removed so that we never recommend a fake hire.	Must	Done
3	As a manager I need a predictive score of post-hire performance trained on the Archive.	Must	Done
4	As the new panel I need public-code, notice-period and title-inflation rules applied so the shortlist reflects this cycle's reality.	Must	Done
5	As Kestrel I need a ranked list of exactly 500 unique candidate_ids ordered most → least likely.	Must	Done
6	As a validator I need the approach checked end-to-end on the Ledger (dev + winners).	Should	Done

4. Sprint Execution (CRISP-DM inside Scrum)

4.1 Sprint 0 – Data Understanding

- Explored train (20k), dev (2.9k), test (10k) schemas and distributions.
- Identified dirty fields: free-text CTC, notice, experience, scores, public_code_contributions.
- Read insider voice note → extracted hard rules (code boost, notice ≤ 60 days, anti-inflation, anti-pedigree).

4.2 Sprint 1 – Data Cleaning & Preprocessing

All cleaning lives in main.py → parsing helpers + clean_and_featurize().

Field / Area	Cleaning / Standardisation Applied
total_experience	Regex parse: "8+ yrs", ">20", "Fresher", "17.6 years" → float years. Recover missing values from career_path durations.
notice_period	"1 month" / "2 months" × 30 → days. "Immediate", "Available now" → 0. "Serving notice - 45 days" → 45.
current_ctc / expected_ctc	Mixed formats (LPA, ₹, \$, INR, absolute) normalised to approximate LPA.
technical_assessment, aptitude_score	Numbers, fractions ("5.7/10"), text ("Outstanding") mapped to 0–100 scale.
public_code_contributions	"not tracked", "~3", "45 merged PRs", "10+" → numeric. NaN / not tracked → 0.
career_path	Extract title-inflation ratio (senior keywords / years). Recover total duration.
Fabrication flags	exp > age–18, extreme title inflation, unreal CTC → flag_inconsistent = 1.
Deduplication	Normalised email + full name; keep highest-scoring instance only.
Categoricals	Label-encode applied_role, recruitment_channel, company_type/size, degree, major. Median impute numerics.

4.3 Sprint 2 – Modelling

- **Algorithm:** LightGBM Regressor (400 trees, lr=0.05, max_depth=7, num_leaves=63).
- **Target:** post_hire_score (0–100) from the Archive.
- **Key features:** tech_score, apt_score, CTC, experience, skills_count, title_inflation, code_contrib, notice_days, role, channel, inconsistency flags.
- **Validation:** Local hold-out + full end-to-end check on Ledger (dev + winners).

4.4 Sprint 3 – Rule Engine & Ranking

Insider-driven adjustments applied after the model prediction:

1. **Public-code boost:** +15 (≥30), +8 (≥15), +3 (≥8) – “dozens of merged contributions get fast-tracked”.
2. **Notice penalty:** –25 if > 60 days – “can’t join inside two months is dead”.
3. **Title-inflation penalty:** –20 / –8 for unrealistic career growth.
4. **Pedigree down-weight:** mild negative for classic top institutes; mild positive for others.
5. **Fabrication filter:** hard remove flag_inconsistent rows.

Final step: sort by adjusted score → de-duplicate → take top 500 → assign ranks 1–500.

5. Definition of Done

- submission.csv contains exactly 500 unique candidate_ids from test.csv, ranked 1 → 500.
- No fabricated profiles and no person listed twice under different IDs.
- Format validated by check_format.py.
- Code is self-contained (main.py) and runs with the permitted library set.
- Documentation covers cleaning, findings, current-cycle differences, exclusions, model and “what did not work”.

6. Acceptance Criteria & Validation

Ledger (dev) end-to-end check (same pipeline, no public-code boost):

Metric	Result
Overlap with true winners in top 150	~33 % (random baseline ≈ 5 %)
Winners appearing in top 300	~70 of 150
Mean / median position of true winners	~681 / ~332

On the Vault shortlist: mean public-code contributions ≈ 40, mean notice ≈ 16 days, zero high title-inflation flags.

7. Sprint Retrospective

What went well

- Robust parsers handled highly inconsistent free-text fields.
- Rule engine directly encoded the insider debrief – largest lift over pure model score.
- Single-file pipeline (main.py) kept the delivery simple and reproducible.

What did not work / abandoned

- Pure rank-by-predicted-score without rules under-performed on the Ledger.
- Heavy reliance on institute prestige or metro city hurt validation (as warned by the insider).
- Aggressive fuzzy deduplication across 10k rows added cost with little extra gain.

Improvements for a future sprint

- Richer career-path sequence modelling.
- Calibrated probability of being in top 5 % instead of regression + rules.
- Persist the trained model artefact for faster re-scoring.

8. How to Run (Definition of Ready → Done)

Prerequisites: Python 3.11+, pandas, numpy, scikit-learn, lightgbm, rapidfuzz. Data files (train.csv, test.csv) in the same directory as main.py.

Command:

```
python main.py
```

Output: submission.csv (ranked 500) + scored_test.csv. Runtime typically < 60 s. Random seed fixed at 42.

9. Deliverable Artifacts

Artifact	Description
submission.csv	Ranked shortlist of 500 candidate_ids (rank 1 = most likely).
code/main.py	Full pipeline: cleaning, features, model, rules, ranking.
documentation.docx / .pdf	This document – Scrum-framed solution write-up.

— End of Documentation —